Kafka Task Queue System

Deep workflow explanation with diagrams

Based on producer.py, consumer.py, docker-compose.yml, and dashboard.html

The one-message question

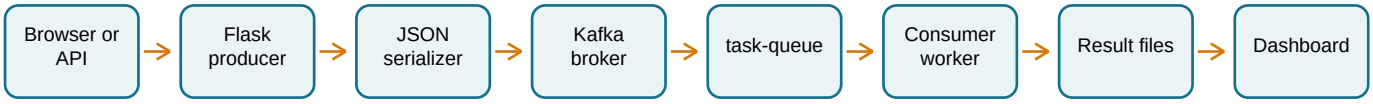
What exactly happens after a user submits one task? This guide follows that task through HTTP, Flask, JSON serialization, Kafka topic and partition, offset, consumer group, worker processing, result persistence, and dashboard refresh.

> Actual configuration: topic task-queue, broker localhost:9092, group task-processor-group, ZooKeeper on 2181, one Docker broker, JSON values, and local JSON files for state.

1. System at a glance

The application has two long-running Python processes. producer.py is both the Flask web server and Kafka producer. consumer.py is the worker and Kafka consumer. They do not call each other directly; Kafka is the transport between them.

Normal message path

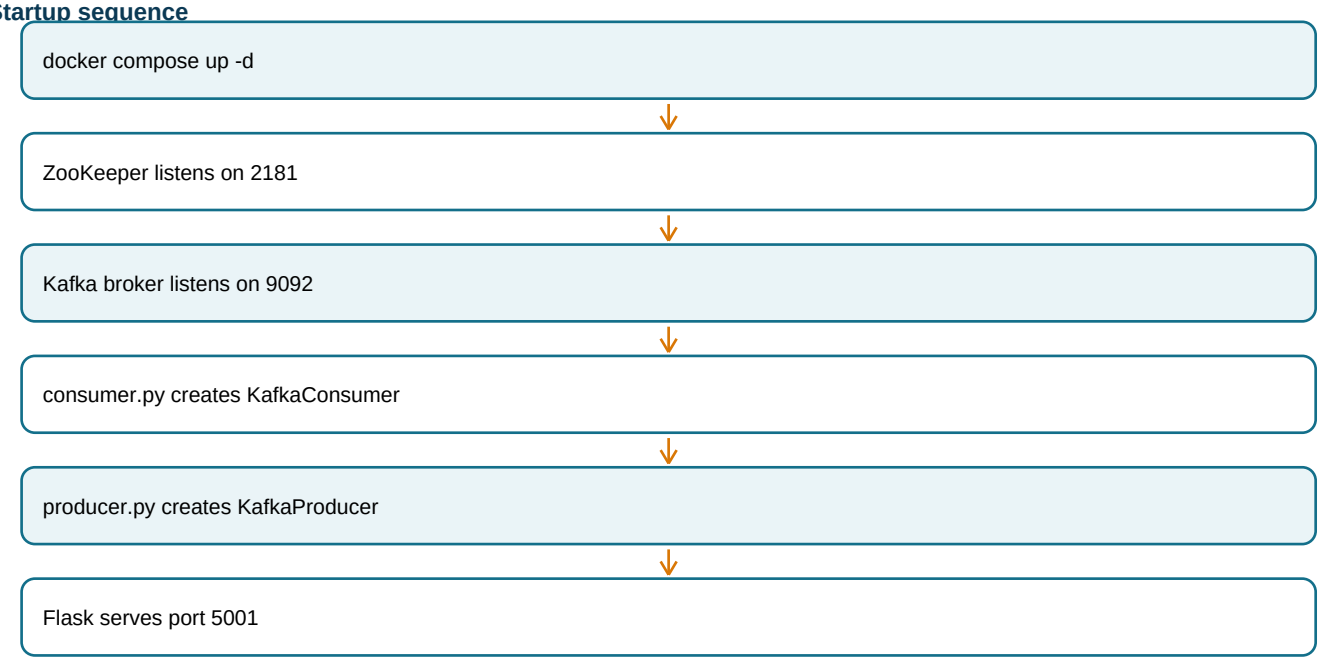


The browser only waits for the enqueue response. The slow work happens later in the consumer. That separation is the reason the dashboard can remain responsive while a task takes 2, 3.5, or 5 seconds.

2. Startup workflow

Docker must provide Kafka before the Python clients can use it. The Compose file starts ZooKeeper first, then the Kafka broker. The producer and consumer both tolerate a broker that is still booting, but their retry behavior is different.

Startup sequence

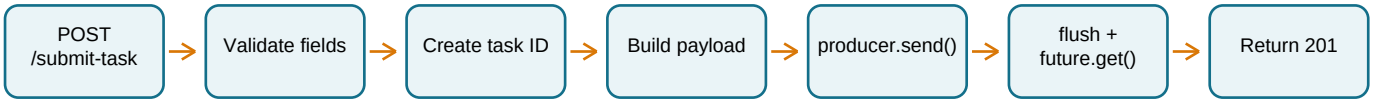


consumer.py tries twice and falls back to standalone queue mode if Kafka is unavailable. producer.py tries to connect during startup and also retries when a request needs a producer. The system can therefore start in a degraded local-file mode, but that mode is not Kafka processing.

3. One task: HTTP to Kafka

A task begins at POST /submit-task. The route accepts JSON or form fields. It requires task_name and normalizes priority to HIGH, MEDIUM, or LOW. Invalid priority values become MEDIUM.

Submission workflow



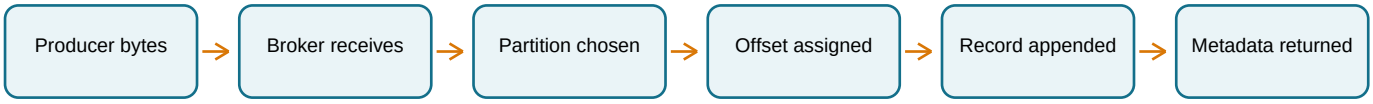
The generated payload contains task_id, task_name, priority, status=QUEUED, submitted_at, and empty completion fields. The value serializer converts the dictionary into JSON UTF-8 bytes. Because no key is passed to send(), the application does not control partition selection with a business key.

> Important timing: the route waits for Kafka metadata for up to the configured timeouts, but it does not wait for the consumer to finish the task.

4. What Kafka does with the record

After producer.send(), the Kafka client sends a record to the broker. Kafka appends it to a partition log. The record receives a partition number and an offset. Those two values identify its position in the topic log.

Broker-side workflow

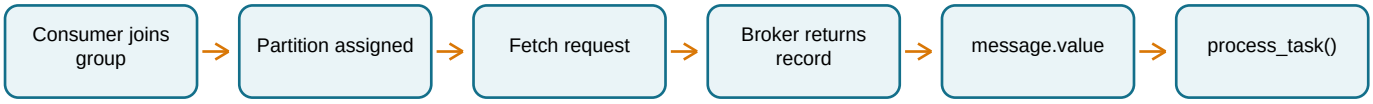


The producer prints the topic, partition, and offset from future.get(). The offset is not the task ID. The task ID is part of the JSON value and is used by the application to match status updates. The offset is Kafka's position marker and is used by the consumer group to track progress.

5. Consumer group and polling

consumer.py creates KafkaConsumer for task-queue with group_id=task-processor-group. The consumer joins the group and receives ownership of the available partition. The for message in consumer loop repeatedly fetches records from Kafka.

Read workflow

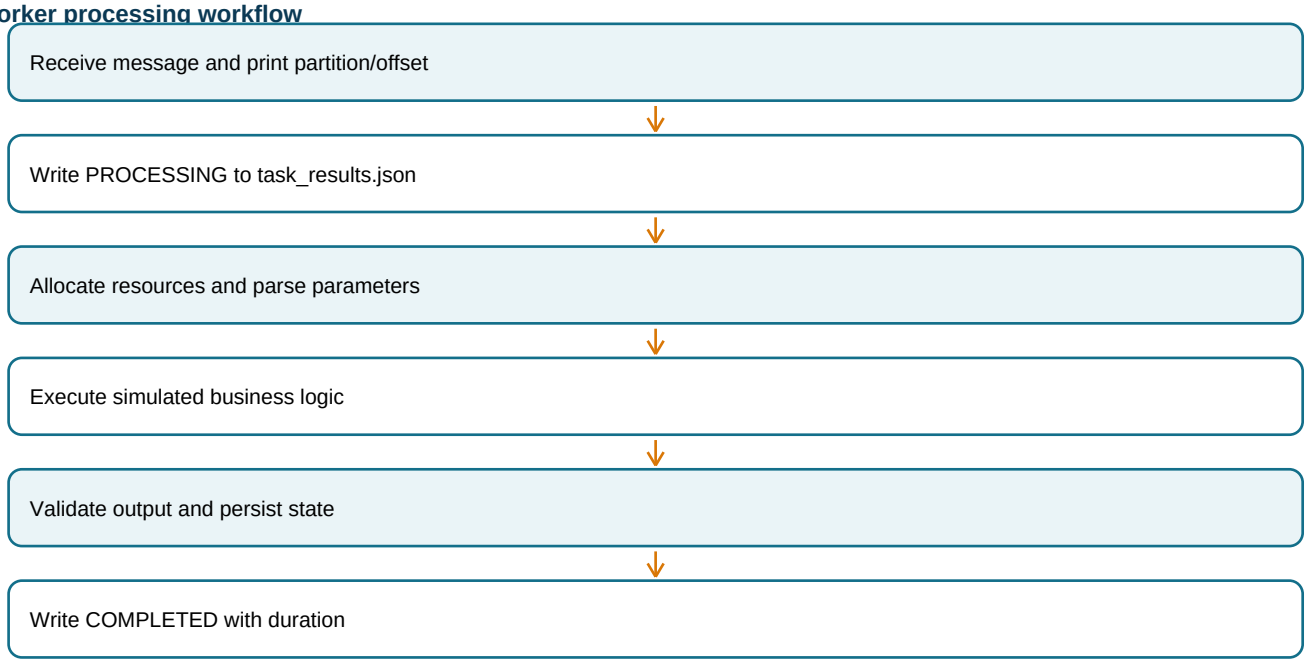


The consumer is pull-based. Kafka does not invoke a Python callback for every message. The client asks Kafka for data, which lets the worker control how quickly it reads and processes work. With one partition, adding more workers to the same group cannot create more than one active owner for that partition.

6. Inside process_task()

The worker deserializes JSON into a Python dictionary before process_task() runs. It reads task_id, task_name, priority, and submitted_at. Priority selects a simulated duration: HIGH 2.0 seconds, MEDIUM 3.5 seconds, LOW 5.0 seconds.

Worker processing workflow

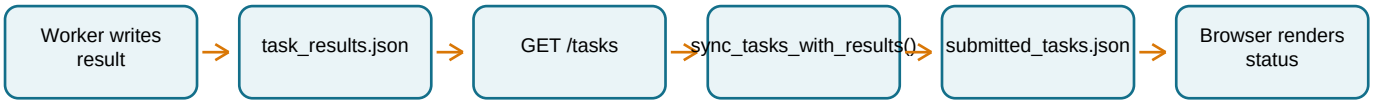


The three sleeps represent stages of real work. They are only a simulation; this project does not call an external business service. The result file is updated atomically by writing a temporary file and replacing the original.

7. How status returns to the dashboard

Kafka carries the task event, but the dashboard does not read Kafka directly. consumer.py writes status into task_results.json. When Flask handles GET / or GET /tasks, producer.py reads that result file and merges matching records into submitted_tasks.json.

Status synchronization

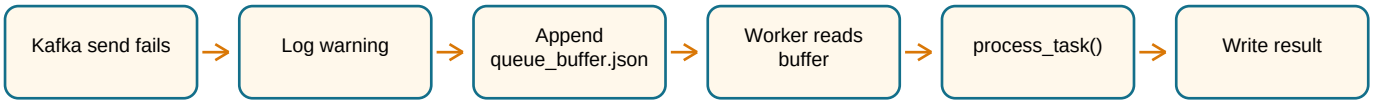


The browser polls GET /tasks every five seconds using fetch(). This produces the visible sequence QUEUED -> PROCESSING -> COMPLETED. The polling delay is a UI refresh interval, not Kafka delivery latency.

8. Failure and fallback workflow

If the producer cannot publish to Kafka, submit_task() writes the task to queue_buffer.json. The consumer checks that file and processes buffered tasks in standalone mode. This keeps the demo usable when Docker is not running, but it is a separate path with weaker durability and concurrency guarantees.

Kafka failure branch

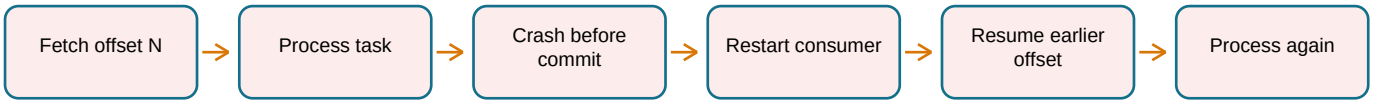


A Kafka send failure does not mean the event was definitely lost or definitely duplicated; the exact outcome depends on where the failure occurred. Production systems normally use explicit retry policies, idempotency keys, durable storage, and dead-letter topics rather than a shared local JSON file.

9. Offset, commit, and duplicates

The consumer uses enable_auto_commit=True with a one-second auto-commit interval. Processing and committing are not one atomic operation. If the worker processes a task and crashes before its offset is committed, Kafka can deliver the same record again after restart.

Possible duplicate path

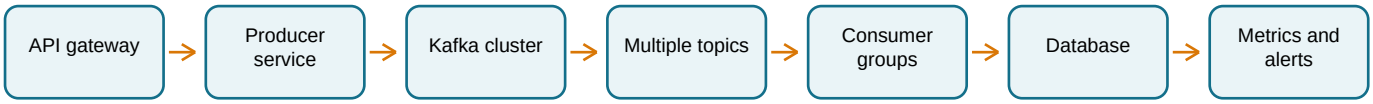


For this reason, the project most closely approaches at-least-once processing. The code does not implement exactly-once processing, a transactional producer, or an idempotent database operation. The task ID can be used as an idempotency key in a stronger design.

10. Production workflow

The learning project is intentionally small: one broker, one topic, local files, PLAINTEXT listeners, and no metrics. A production workflow adds durable databases, multiple brokers, replication, explicit partitions, secure listeners, schema governance, monitoring, retries, and dead-letter handling.

Production evolution

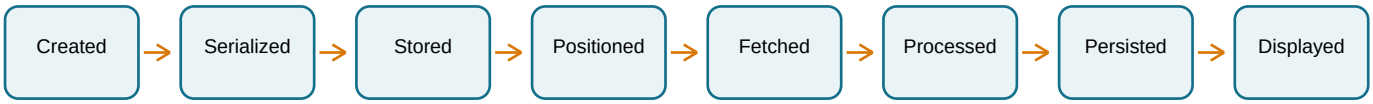


The core message path does not change. What changes is the reliability around it: the broker cluster survives failure, consumers can scale across partitions, handlers can safely retry, and operators can see lag and errors before users report them.

11. Exact answer: what happens to one message?

1. A user submits task_name and priority. 2. Flask validates input and creates TASK-XXXXXXXX. 3. producer.py creates a Python dictionary. 4. kafka-python serializes it to JSON bytes. 5. Kafka appends it to task-queue. 6. Kafka assigns partition and offset. 7. task-processor-group fetches the record. 8. consumer.py deserializes JSON. 9. process_task() writes PROCESSING. 10. The worker performs three simulated stages. 11. It writes COMPLETED and duration. 12. Flask merges the result. 13. The dashboard shows the new state on its next poll.

One-message mental model



The most important separation is between transport state and business state. Kafka stores the event and tracks consumer progress. task_results.json stores this demo's business status. In a larger system, a database or service would own business state, while Kafka would remain the event transport.

12. Debugging by workflow stage

Debug in the same order as the message moves. First verify Docker and broker connectivity. Then verify the topic. Next inspect producer logs for send, partition, and offset. Then inspect consumer logs for the Kafka event and processing stages. Finally inspect task_results.json and GET /tasks.

Stage	Evidence to check
Infrastructure	docker compose ps; port 9092; ZooKeeper 2181
Producer	POST response; send exception; topic/partition/offset log
Consumer	group name; Kafka event; processing logs
State	task_results.json and submitted_tasks.json
UI	GET /tasks and five-second browser polling

This stage-by-stage method prevents guessing. A task visible in submitted_tasks.json but absent from Kafka logs points to a publish or fallback issue. A Kafka event with no result points to consumer or processing failure. A completed result with an unchanged browser points to the status synchronization or UI polling layer.